

DEPARTMENT OF STATISTICS, PONDICHERRY UNIVERSITY
M.Sc. Quantitative Finance — Semester III
MSQF 536: MACHINE LEARNING TECHNIQUES
UNIT I ASSIGNMENT ANSWERS: DATA PRE-PROCESSING

Student Name: N Rohit Vedhanandh
Programme: M.Sc. Quantitative Finance

Course Code: MSQF 536
Topic: Unit I Assignment Answers

Section A: Conceptual Questions

[10 Marks]

1. Define Machine Learning and Explain Data Pre-processing Importance

Question: Define Machine Learning in your own words. Explain why data pre-processing is an important stage in a machine learning workflow.

Answer:

Machine Learning is a subset of artificial intelligence that enables systems to automatically learn and improve from experience without being explicitly programmed. It involves algorithms that identify patterns in data and make predictions or decisions based on those patterns.

Why Data Pre-processing is Important:

- **Improves data quality:** Raw data often contains noise, missing values, and inconsistencies
- **Enhances model performance:** Properly preprocessed data leads to better accuracy and generalization
- **Reduces bias:** Removes systematic errors and inconsistencies
- **Ensures algorithm compatibility:** Many algorithms require specific input formats
- **Handles scale differences:** Prevents features with larger magnitudes from dominating
- **Manages missing values:** Prevents loss of valuable information

2. Types of Variables with Financial Examples

Question: Distinguish between the following types of variables and give one suitable financial example for each: (a) Continuous variable, (b) Discrete variable, (c) Nominal variable, (d) Ordinal variable, (e) Binary variable.

Answer:

(a) Continuous Variable

- **Definition:** Can take any value within a range, including decimals
- **Example:** Stock price (Rs. 145.67, Rs. 145.68, Rs. 145.69...)

(b) Discrete Variable

- **Definition:** Takes only specific, countable values
- **Example:** Number of trades in a day (150, 151, 152...)

(c) Nominal Variable

- **Definition:** Categorical with no inherent order
- **Example:** Stock exchange (NSE, BSE, NYSE)

(d) Ordinal Variable

- **Definition:** Categorical with inherent order/ranking
- **Example:** Credit rating (AAA > AA > A > BBB)

(e) Binary Variable

- **Definition:** Only two possible values
- **Example:** Loan default status (Default = 1, No Default = 0)

3. Numerical vs. Categorical Variables

Question: Explain the difference between numerical and categorical variables. Why is this distinction important in data pre-processing?

Answer:

Numerical Variables: Represent quantities that can be measured or counted (e.g., stock price, trading volume)

Categorical Variables: Represent categories or groups without quantitative meaning (e.g., sector, stock name)

Importance of Distinction:

- **Processing methods differ:** Numerical variables need scaling; categorical need encoding
- **Algorithm requirements:** Some algorithms (e.g., regression) require numerical input
- **Statistical treatment:** Different summary statistics for each type
- **Interpretation:** Different meaning and handling in models

4. Data Pre-processing Pipeline Steps

Question: Explain the following steps in a data pre-processing pipeline: (a) Cleaning, (b) Transformation, (c) Normalization/Scaling, (d) Encoding.

Answer:**(a) Cleaning**

- Removing duplicates
- Handling missing values
- Correcting inconsistencies
- Removing irrelevant features

(b) Transformation

- Converting data types
- Creating derived features
- Applying mathematical transformations (log, square root)
- Feature engineering

(c) Normalization/Scaling

- **Normalization** (Min-Max): Rescaling to [0, 1] range
- **Standardization** (Z-score): Scaling to zero mean, unit variance
- Purpose: Bring features to comparable scales

(d) Encoding

- Converting categorical variables to numerical
- Methods: One-hot encoding, label encoding

- Purpose: Make data compatible with machine learning algorithms

5. Purpose of Datasets

Question: Explain the purpose of the following datasets: (a) Training set, (b) Validation set, (c) Testing set.

Answer:

(a) Training Set

- Used to train the model by learning patterns and relationships
- Typically 60-80% of total data
- Model learns parameters from this set

(b) Validation Set

- Used to tune hyperparameters
- Evaluates model performance during development
- Helps prevent overfitting through model selection

(c) Testing Set

- Used only once for final evaluation
- Provides unbiased assessment of model performance
- Simulates real-world data the model hasn't seen

Section B: Numerical and Short Answer Questions [10 Marks]

1. Variable Classification

Question: Consider the following variables from a financial dataset:

$\{\text{Stock Price, Number of Trades, Sector, Credit Rating, Loan Default}\}$

Classify each variable as continuous, discrete, nominal, ordinal, or binary.

Answer:

Variable	Type
Stock Price	Continuous
Number of Trades	Discrete
Sector	Nominal
Credit Rating	Ordinal
Loan Default	Binary

2. Calculate Mean for Missing Value Imputation

Question: The observed daily trading volumes of a stock are: 100, 120, 140, 160, 180. Calculate the mean value that would be used to replace a missing observation.

Answer:

Trading volumes: 100, 120, 140, 160, 180

$$\text{Mean} = (100 + 120 + 140 + 160 + 180)/5$$

$$\text{Mean} = 700 / 5 = 140$$

The missing observation would be replaced with 140.

3. Min-Max Scaling Calculation

Question: For $x = 80$, $x_{\min} = 20$, $x_{\max} = 100$, calculate the Min-Max scaled value using $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$.

Answer:

$$\text{Given: } x = 80, x_{\min} = 20, x_{\max} = 100$$

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

$$x' = (80 - 20) / (100 - 20)$$

$$x' = 60/80$$

$$\mathbf{x' = 0.75}$$

4. Z-Score Calculation

Question: A variable has mean $\mu = 50$ and standard deviation $\sigma = 10$. Calculate the Z-score when $x = 70$ using $z = \frac{x - \mu}{\sigma}$.

Answer:

$$\text{Given: } \mu = 50, \sigma = 10, x = 70$$

$$z = (x - \mu) / \sigma$$

$$z = (70 - 50) / 10$$

$$z = 20/10$$

$$\mathbf{z = 2}$$

This means the value of 70 is 2 standard deviations above the mean.

5. 80-20 Train-Test Split

Question: A dataset contains 10,000 observations. If an 80–20 train-test split is used, determine the number of observations in the training and testing sets.

Answer:

$$\text{Total observations} = 10,000$$

$$\text{Training set} = 80\% \text{ of } 10,000 = 10,000 \times 0.80 = \mathbf{8,000}$$

$$\text{Testing set} = 20\% \text{ of } 10,000 = 10,000 \times 0.20 = \mathbf{2,000}$$

6. Problem with Random Train-Test Split for Time-Series Data

Question: Explain why a random train-test split may be problematic for daily stock-price time-series data.

Answer:

Why random split is problematic:

1. **Temporal Order Violation:** Stock prices have temporal dependencies; past data influences future
2. **Look-ahead Bias:** Random split may place future data in training and past data in testing
3. **Information Leakage:** Future information could inadvertently affect past predictions
4. **Time Dependencies:** Auto-correlation means observations are not independent

Solution: Use **time-based split** where training data is from an earlier period and test data from a later period.

7. Confusion Matrix Calculations

Question: Consider the following confusion matrix:

	Predicted Positive	Predicted Negative
Actual Positive	40	5
Actual Negative	10	945

Calculate: (a) Accuracy, (b) Precision, (c) Recall, (d) F1-score.

Answer:

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	40	5
Actual Negative	10	945

Calculations:

$$\begin{aligned}
 \text{(a) Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\
 &= (40 + 945) / (40 + 945 + 10 + 5) \\
 &= 985 / 1000 \\
 &= \mathbf{0.985 \text{ or } 98.5\%}
 \end{aligned}$$

$$\begin{aligned}
 \text{(b) Precision} &= TP / (TP + FP) \\
 &= 40 / (40 + 10) \\
 &= 40 / 50 \\
 &= \mathbf{0.80 \text{ or } 80\%}
 \end{aligned}$$

$$\begin{aligned}
 \text{(c) Recall} &= TP / (TP + FN) \\
 &= 40 / (40 + 5) \\
 &= 40 / 45 \\
 &= \mathbf{0.889 \text{ or } 88.9\%}
 \end{aligned}$$

$$\begin{aligned}
 \text{(d) F1-Score} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \\
 &= 2 \times (0.80 \times 0.889) / (0.80 + 0.889) \\
 &= 2 \times 0.711 / 1.689 \\
 &= 1.422 / 1.689 \\
 &= \mathbf{0.842 \text{ or } 84.2\%}
 \end{aligned}$$

8. Why Accuracy is Misleading for Imbalanced Targets

Question: Explain why accuracy alone may be misleading when the target variable is highly imbalanced.

Answer:

Reasons:

1. **False Sense of Performance:** A model predicting the majority class for all instances can achieve high accuracy
2. **Example:** In fraud detection with 99% non-fraud cases:
 - Model predicting “No Fraud” for everything = 99% accuracy
 - But it fails to catch any actual fraud!
3. **Misguided Business Decisions:** High accuracy may hide poor performance on minority class
4. **Better Metrics:** For imbalanced problems, use:
 - Precision, Recall, F1-Score
 - AUC-ROC
 - Matthews Correlation Coefficient (MCC)

Section C: Long Answer Questions [12 Marks - Answer Any Four]**1. MCAR, MAR, MNAR Missing-Data Mechanisms**

Question: *Explain the difference between MCAR, MAR, and MNAR missing-data mechanisms. Give one financial example for each.*

Answer:**Missing Completely At Random (MCAR)**

- Missingness is completely random and independent of all variables
- Example: A stock price recording fails due to random system glitch

Missing At Random (MAR)

- Missingness depends on observed variables, not the missing value
- Example: A credit score missing more often for customers with lower income (income is observed)

Missing Not At Random (MNAR)

- Missingness depends on the unobserved value itself
- Example: High-income individuals less likely to report their income (the missingness depends on the value being high)

2. Missing-Value Imputation Methods

Question: *Compare the following missing-value imputation methods: (a) Mean/Median imputation, (b) Mode imputation, (c) KNN imputation, (d) Forward fill, (e) Backward fill, (f) Interpolation.*

Answer:**(a) Mean/Median Imputation**

- **Pros:** Simple, fast, works for numerical variables
- **Cons:** Reduces variance, may distort distribution, ignores relationships

(b) Mode Imputation

- **Pros:** Simple, suitable for categorical variables
- **Cons:** Can introduce bias, may alter frequency distributions

(c) KNN Imputation

- **Pros:** Uses similarity with other observations, captures relationships
 - **Cons:** Computationally expensive, sensitive to K parameter
- (d) **Forward Fill**
- **Pros:** Works well for time series, maintains temporal order
 - **Cons:** Assumes last value persists, may propagate errors
- (e) **Backward Fill**
- **Pros:** Uses future known values for missing data
 - **Cons:** May introduce look-ahead bias
- (f) **Interpolation**
- **Pros:** Smoothly estimates missing values in time series
 - **Cons:** Assumes linear/continuous relationships, may not capture sudden changes

3. Normalization vs. Standardization

Question: Explain the difference between normalization and standardization. Give the mathematical formula for each.

Answer:

Normalization (Min-Max Scaling)

- Purpose: Transform features to a fixed range [0,1]
- Formula: $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$
- Best when: Data distribution is not Gaussian, need bounded values
- Sensitive to outliers

Standardization (Z-Score)

- Purpose: Scale features to zero mean and unit variance
- Formula: $z = (x - \mu) / \sigma$
- Best when: Data is normally distributed, need unbounded values
- Less sensitive to outliers

When to use:

- Normalization: Neural networks, KNN, SVM
- Standardization: Linear regression, Logistic regression, PCA

4. One-Hot Encoding

Question: Explain one-hot encoding. Consider the categorical variable:

$$\text{Sector} \in \{\text{Banking}, \text{IT}, \text{Energy}, \text{Pharma}\}$$

Show how this variable can be represented using one-hot encoding.

Answer:

Explanation: One-hot encoding converts categorical variables into binary vectors. Each category becomes a separate binary column where 1 indicates presence and 0 indicates absence.

Example: Original variable: Sector $\in$ [Banking, IT, Energy, Pharma]

Original	Banking	IT	Energy	Pharma
Banking	1	0	0	0
IT	0	1	0	0
Energy	0	0	1	0
Pharma	0	0	0	1

Advantages:

- No ordinal relationship assumed
- All categories equally weighted
- Compatible with most algorithms

Disadvantages:

- Can create too many features with high cardinality
- May cause multicollinearity if all categories used

5. Why KNN and SVM are Sensitive to Scale

Question: Explain why KNN and SVM are sensitive to the scale of variables. Illustrate your explanation with a simple financial example.

Answer:

Sensitivity Explanation: KNN (K-Nearest Neighbors):

- Uses distance metrics (Euclidean, Manhattan) to find neighbors
- Features with larger scales dominate distance calculations
- Small values become insignificant in comparison

SVM (Support Vector Machines):

- Uses distance from decision boundary
- Features with larger scales dominate the optimization process
- Kernel computations scale with feature magnitudes

Financial Example:

Customer	Age	Income (\$)	Days Delinquent
A	25	50,000	10
B	30	100,000	5
C	35	150,000	0

Without scaling:

- Euclidean distance dominated by income difference
- Age and Days Delinquent contribute negligibly
- KNN would essentially use only income to find neighbors

With scaling:

- All features contribute equally to distance
- More balanced representation of customer characteristics

6. Over-sampling vs. Under-sampling and SMOTE

Question: *Explain the difference between over-sampling and under-sampling. What is SMOTE, and how does it differ from simple duplication of minority observations?*

Answer:

Over-sampling:

- Increases minority class samples to balance the dataset
- May lead to overfitting
- Simple duplication creates identical copies

Under-sampling:

- Reduces majority class samples to balance the dataset
- May lose valuable information
- Can discard important patterns from majority class

SMOTE (Synthetic Minority Over-sampling Technique):

- Creates synthetic minority samples using interpolation
- For each minority sample, find k nearest neighbors
- Create new samples along the line connecting them

Difference from Simple Duplication:

- SMOTE creates **new synthetic examples**, not exact copies
- SMOTE increases variety in minority class
- Duplication just repeats existing examples
- SMOTE reduces overfitting risk compared to simple duplication

7. Data Leakage and Pre-processing

Question: *Explain the concept of data leakage. Why is it incorrect to scale or impute the complete dataset before performing the train-test split?*

Answer:

Data Leakage Definition: Data leakage occurs when information from outside the training dataset is used to create the model, causing overly optimistic performance estimates.

Why Not to Scale/Impute Before Train-Test Split:

1. Information Leakage:

- Statistics (mean, min, max) from entire dataset include test data
- Model “sees” test data patterns during training

2. Unrealistic Performance:

- Model appears to perform better than it actually would
- Real-world performance is worse

3. Example:

- If we scale before splitting, we use test data min/max to scale training data
- Training process has access to test data statistics

Correct Approach:

1. Split data first (train/validation/test)
2. Calculate scaling parameters ONLY on training set
3. Apply scaling to validation and test sets using training parameters
4. Impute missing values using training set statistics only

8. Fraud Detection as Class Imbalance Problem

Question: Explain why fraud detection is a typical class-imbalance problem. Discuss why a model predicting “No Fraud” for every transaction may still achieve very high accuracy.

Answer:**Why Fraud Detection is Imbalanced:**

1. **Nature of the Problem:**
 - Fraudulent transactions are rare ($\sim 0.1\text{--}1\%$)
 - Most transactions are legitimate
2. **Dataset Characteristics:**
 - Majority class: Non-fraud (99%+)
 - Minority class: Fraud ($< 1\%$)

Why “No Fraud” Model May Achieve High Accuracy:

Prediction	Actual: Non-Fraud	Actual: Fraud
Non-Fraud	99,000 (TP)	100 (FN)
Fraud	0 (FP)	0 (TN)

$$\text{Accuracy} = (99,000 + 0) / (99,000 + 0 + 0 + 100) = 99.9\%$$

Why This Model is Useless:

- Fails to detect ALL fraud cases
- Misses 100 fraudulent transactions
- Financial loss from undetected fraud
- False sense of security

Better Approach:

- Use Precision, Recall, F1-Score
- Consider cost-sensitive learning
- Apply SMOTE or other balancing techniques
- Use anomaly detection methods

Section D: Python Exercises**[10 Marks]****1. Identifying Variable Types**

Question: Create a Pandas DataFrame containing the following variables: stock, sector, credit_rating, close_price, num_trades. Using Python: (a) Display the DataFrame, (b) Display the data types, (c) Identify the categorical variables, (d) Identify the numerical variables.

Answer:

```
import pandas as pd
import numpy as np

# Create DataFrame
data = {
    'stock': ['AAPL', 'GOOGL', 'MSFT', 'AMZN', 'TSLA'],
    'sector': ['Technology', 'Technology', 'Technology', 'E-commerce', 'Automotive'],
    'credit_rating': ['AA', 'AAA', 'AA+', 'A+', 'BBB+'],
    'close_price': [175.34, 140.56, 330.45, 165.23, 245.67],
    'num_trades': [1500, 1200, 980, 2100, 1650]
}

df = pd.DataFrame(data)

# 1) Display the DataFrame
print("DataFrame:")
print(df)
print("\n")

# 2) Display the data types
print("Data Types:")
print(df.dtypes)
print("\n")

# 3) Identify categorical variables
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
print("Categorical Variables:", categorical_cols)

# 4) Identify numerical variables
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
print("Numerical Variables:", numerical_cols)
```

2. Scaling and Encoding

Question: Create a DataFrame containing a numerical variable close_price and a categorical variable sector. Using Python: (a) Apply Min-Max scaling to close_price, (b) Apply Z-score standardization to close_price, (c) Apply one-hot encoding to sector, (d) Display the resulting DataFrame.

Answer:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.preprocessing import OneHotEncoder

# Create DataFrame
data = {
    'close_price': [100, 250, 150, 300, 200, 50, 180, 400],
    'sector': ['Banking', 'IT', 'IT', 'Pharma', 'Banking', 'IT', 'Pharma', 'Banking']
}

# (a) Min-Max scaling
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data[['close_price']])

# (b) Z-score standardization
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data[['close_price']])

# (c) One-hot encoding
encoder = OneHotEncoder()
data_encoded = encoder.fit_transform(data[['sector']])

# (d) Display the resulting DataFrame
print(data_scaled)
print(data_standardized)
print(data_encoded.toarray())
```

```

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print("\n")

# (a) Min-Max scaling
scaler_minmax = MinMaxScaler()
df['close_price_minmax'] = scaler_minmax.fit_transform(df[['close_price']])

# (b) Z-score standardization
scaler_std = StandardScaler()
df['close_price_zscore'] = scaler_std.fit_transform(df[['close_price']])

print("After Scaling:")
print(df)
print("\n")

# (c) One-hot encoding
one_hot = pd.get_dummies(df['sector'], prefix='sector')
df_encoded = pd.concat([df, one_hot], axis=1)
df_encoded = df_encoded.drop('sector', axis=1)

# (d) Display resulting DataFrame
print("After One-Hot Encoding:")
print(df_encoded)

```

3. Missing-Value Imputation

Question: Create a DataFrame containing at least two numerical variables with missing observations. Using Python: (a) Identify the missing values, (b) Perform mean imputation, (c) Perform median imputation, (d) Perform KNN imputation, (e) Compare the results obtained from the three methods.

Answer:

```

import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer

# Create DataFrame with missing values
np.random.seed(42)
data = {
    'stock_price': [100, np.nan, 150, 200, np.nan, 180, 220, 160, 190, np.
nan],
    'trading_volume': [1000, 1200, np.nan, 1500, 1300, np.nan, 1100, 1400,
1600, 1700],
    'return_ratio': [0.05, 0.03, 0.08, np.nan, 0.02, 0.07, 0.04, 0.06, np.
nan, 0.09]
}

df = pd.DataFrame(data)
print("Original DataFrame with Missing Values:")
print(df)
print("\n")

# (a) Identify missing values
print("Missing Values per Column:")
print(df.isnull().sum())
print("\n")

# (b) Mean imputation

```

```

df_mean = df.copy()
df_mean['stock_price'] = df_mean['stock_price'].fillna(df_mean['stock_price'].mean())
df_mean['trading_volume'] = df_mean['trading_volume'].fillna(df_mean['trading_volume'].mean())
df_mean['return_ratio'] = df_mean['return_ratio'].fillna(df_mean['return_ratio'].mean())
print("Mean Imputation:")
print(df_mean)
print("\n")

# (c) Median imputation
df_median = df.copy()
df_median['stock_price'] = df_median['stock_price'].fillna(df_median['stock_price'].median())
df_median['trading_volume'] = df_median['trading_volume'].fillna(df_median['trading_volume'].median())
df_median['return_ratio'] = df_median['return_ratio'].fillna(df_median['return_ratio'].median())
print("Median Imputation:")
print(df_median)
print("\n")

# (d) KNN imputation
knn_imputer = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(
    knn_imputer.fit_transform(df),
    columns=df.columns
)
print("KNN Imputation:")
print(df_knn)
print("\n")

# (e) Compare results
print("Comparison of Imputation Methods (Stock Price):")
print(f"Original missing values: {df['stock_price'].isnull().sum()}")
print(f"Mean imputation values: {df_mean['stock_price'].tolist()}")
print(f"Median imputation values: {df_median['stock_price'].tolist()}")
print(f"KNN imputation values: {df_knn['stock_price'].tolist()}")

```

4. Train-Test Split

Question: Using a suitable financial dataset, perform an 80-20 train-test split. Report: (a) Number of observations in the original dataset, (b) Number of observations in the training set, (c) Number of observations in the testing set, (d) The Python code used.

Answer:

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# Create a financial dataset
np.random.seed(42)
n_samples = 1000

data = {
    'feature1': np.random.randn(n_samples),
    'feature2': np.random.randn(n_samples),
    'feature3': np.random.randn(n_samples),
    'target': np.random.randint(0, 2, n_samples)
}

```

```

}

df = pd.DataFrame(data)

# (a) Number of observations in original dataset
print(f"Original dataset observations: {len(df)}")
print(f"Original dataset shape: {df.shape}")
print("\n")

# Perform 80-20 train-test split
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y # Ensures class distribution is maintained
)

# (b) Number of observations in training set
print(f"Training set observations: {len(X_train)}")

# (c) Number of observations in testing set
print(f"Testing set observations: {len(X_test)}")

# (d) Python code used (shown above)
print("\nTrain-Test Split Code:")
print("""
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
""")

```

5. Class Imbalance

Question: Using a suitable binary classification dataset: (a) Display the class distribution before balancing, (b) Apply an appropriate sampling technique, (c) Display the class distribution after balancing, (d) Compare the class distributions before and after balancing, (e) Explain why the balancing procedure should not be applied to the test set.

Answer:

```

import pandas as pd
import numpy as np
from imblearn.over_sampling import SMOTE
from sklearn.datasets import make_classification

# Create imbalanced dataset
X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=10,
    n_redundant=5,
    n_classes=2,
    weights=[0.9, 0.1], # 90% class 0, 10% class 1
    random_state=42
)

```

```

)

df = pd.DataFrame(X)
df['target'] = y

# (a) Class distribution before balancing
print("Class Distribution BEFORE Balancing:")
print(df['target'].value_counts())
print(f"Class 0: {df['target'].value_counts()[0]} ({df['target'].value_counts()[0]/len(df)*100:.2f}%)")
print(f"Class 1: {df['target'].value_counts()[1]} ({df['target'].value_counts()[1]/len(df)*100:.2f}%)")
print("\n")

# (b) Apply SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Create balanced DataFrame
df_balanced = pd.DataFrame(X_resampled)
df_balanced['target'] = y_resampled

# (c) Class distribution after balancing
print("Class Distribution AFTER Balancing:")
print(df_balanced['target'].value_counts())
print(f"Class 0: {df_balanced['target'].value_counts()[0]} ({df_balanced['target'].value_counts()[0]/len(df_balanced)*100:.2f}%)")
print(f"Class 1: {df_balanced['target'].value_counts()[1]} ({df_balanced['target'].value_counts()[1]/len(df_balanced)*100:.2f}%)")
print("\n")

# (d) Compare class distributions
print("Comparison of Class Distributions:")
print(f"{'Metric':<20} {'Before':<15} {'After':<15}")
print("-" * 50)
print(f"{'Total samples':<20} {len(df):<15} {len(df_balanced):<15}")
print(f"{'Minority class':<20} {df['target'].value_counts()[1]:<15} {df_balanced['target'].value_counts()[1]:<15}")
print(f"{'Majority class':<20} {df['target'].value_counts()[0]:<15} {df_balanced['target'].value_counts()[0]:<15}")
print(f"{'Imbalance ratio':<20} {df['target'].value_counts()[0]/df['target'].value_counts()[1]:<15.2f} {df_balanced['target'].value_counts()[0]/df_balanced['target'].value_counts()[1]:<15.2f}")

# (e) Why balancing should not be applied to test set
print("\nWhy NOT to balance the test set:")
print("""
1. Test set should represent real-world data distribution
2. Balancing test set would create unrealistic evaluation
3. Model performance should be evaluated on original data distribution
4. Real-world application will have imbalanced data
5. Balancing test set would make results non-comparable with other models
6. It would hide the model's true performance on minority class
""")

```

Section E: Finance-Based Case Study

[8 Marks]

Loan Default Prediction: Data Pre-processing Workflow

A financial institution has collected historical customer information to develop a machine learning model for predicting loan default. The dataset contains the following variables:

Variable	Description
Age	Customer age
Income	Annual income
Credit Score	Credit score
Employment Type	Employment category
Credit Rating	Rating category
Loan Amount	Amount of loan
Debt-to-Income Ratio	Ratio of debt to income
Previous Default	Previous default status
Loan Default	Target variable

1. Identify Numerical and Categorical Variables

Question: Identify the numerical and categorical variables.

Answer:

Numerical Variables:

- Age
- Income
- Credit Score
- Loan Amount
- Debt-to-Income Ratio

Categorical Variables:

- Employment Type (Nominal)
- Credit Rating (Ordinal)
- Previous Default (Binary)

Target Variable:

- Loan Default (Binary)

2. Encoding Methods for Categorical Variables

Question: Suggest appropriate encoding methods for the categorical variables.

Answer:

Variable	Encoding Method	Reason
Employment Type	One-Hot Encoding	No inherent order, categories are mutually exclusive
Credit Rating	Ordinal Encoding	Has natural order (AAA > AA > A > BBB)
Previous Default	Binary Encoding	Only two values (Default/No Default)

Python Implementation:

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

# One-hot encoding for Employment Type
one_hot = OneHotEncoder(sparse_output=False)
employment_encoded = one_hot.fit_transform(df[['Employment_Type']])

# Ordinal encoding for Credit Rating
rating_order = ['BBB', 'BBB+', 'A-', 'A', 'A+', 'AA-', 'AA', 'AA+', 'AAA']
ordinal = OrdinalEncoder(categories=[rating_order])
credit_encoded = ordinal.fit_transform(df[['Credit_Rating']])
```

3. Handling Missing Values

Question: Suggest suitable methods for handling missing values in the numerical and categorical variables.

Answer:**Numerical Variables:**

- **Age:** Median imputation (handles outliers better)
- **Income:** Median imputation or KNN imputation
- **Credit Score:** KNN imputation (relationship with other variables)
- **Loan Amount:** Mean imputation
- **DTI Ratio:** Forward fill if time series, else median

Categorical Variables:

- **Employment Type:** Mode imputation
- **Credit Rating:** Forward fill (if ordered) or create “Missing” category
- **Previous Default:** Mode imputation or treat as separate category

Python Implementation:

```
from sklearn.impute import SimpleImputer, KNNImputer

# Numerical - Median imputation
num_imputer = SimpleImputer(strategy='median')
df_numerical = num_imputer.fit_transform(df[numerical_cols])

# Numerical - KNN imputation
knn_imputer = KNNImputer(n_neighbors=5)
df_numerical_knn = knn_imputer.fit_transform(df[numerical_cols])

# Categorical - Mode imputation
cat_imputer = SimpleImputer(strategy='most_frequent')
df_categorical = cat_imputer.fit_transform(df[categorical_cols])
```

4. Scaling Numerical Variables

Question: Explain how the numerical variables should be scaled.

Answer:**Recommended Approach:**

1. **For Tree-based models** (Random Forest, XGBoost):

- No scaling needed
 - These models are invariant to scale
2. **For Distance-based models** (KNN, SVM, Logistic Regression):
- Standardization (Z-score): For variables with normal distribution
 - Normalization (Min-Max): For variables with skewed distribution

Implementation:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Standardization
scaler_std = StandardScaler()
df_standardized = scaler_std.fit_transform(df[numerical_cols])

# Normalization
scaler_minmax = MinMaxScaler()
df_normalized = scaler_minmax.fit_transform(df[numerical_cols])
```

5. Train-Validation-Test Strategy

Question: *Propose a suitable train-validation-test strategy.*

Answer:

Proposed Strategy:

1. **Time-Based Split** (if temporal data exists):
 - Training: 70% (earlier period)
 - Validation: 15% (middle period)
 - Test: 15% (latest period)
2. **Stratified Split** (if cross-sectional data):
 - Training: 70% (with class stratification)
 - Validation: 15% (with class stratification)
 - Test: 15% (with class stratification)

Implementation:

```
from sklearn.model_selection import train_test_split

# First split: Train vs (Validation + Test)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Second split: Validation vs Test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5,
    random_state=42,
    stratify=y_temp
)
```

6. Handling Class Imbalance

Question: Explain how you would handle the class imbalance in the target variable.

Answer:

Recommended Strategy:

1. **Assess Imbalance Ratio:** Check if default cases < 5% of total
2. **Model Selection:**
 - Use algorithms with class weights (Random Forest, XGBoost, Logistic Regression)
 - Set `class_weight='balanced'` or calculate weights manually
3. **Resampling Techniques:**
 - **SMOTE:** Create synthetic minority samples
 - **ADASYN:** Adaptive synthetic sampling
 - **ROSE:** Random over-sampling examples
4. **Ensemble Methods:** Balanced Random Forest, Easy Ensemble, RUSBoost

Implementation:

```
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier

# Apply SMOTE only on training data
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)

# Use class weights
rf = RandomForestClassifier(
    class_weight='balanced',
    random_state=42
)

# Or custom weights
class_weights = {
    0: 1, # Non-default
    1: len(y_train[y_train==0]) / len(y_train[y_train==1]) # Default
}
rf = RandomForestClassifier(class_weight=class_weights)
```

7. Possible Sources of Data Leakage

Question: Identify two possible sources of data leakage in this problem.

Answer:

Source 1: Future Information in Training

- **Example:** Using future financial data (e.g., post-loan performance) to predict default
- **Solution:** Use only data available before loan approval

Source 2: Scaling Before Train-Test Split

- **Example:** Scaling entire dataset before splitting
- **Solution:** Calculate scaling parameters only on training set

Source 3: Feature Engineering with Target Information

- **Example:** Creating features using target variable
- **Solution:** Ensure features use only pre-loan information

Source 4: Resampling Before Split

- **Example:** Applying SMOTE before train-test split
- **Solution:** Apply resampling after splitting, only on training data

Source 5: Temporal Data Leakage

- **Example:** Using future data to fill missing values
- **Solution:** Use forward fill or impute using only past data

8. Step-by-Step Data Pre-processing Workflow

Question: Prepare a step-by-step data pre-processing workflow for this dataset before applying a machine learning classification algorithm.

Answer:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    OrdinalEncoder
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE

# Step 1: Load and Explore Data
df = pd.read_csv('loan_data.csv')
print("Dataset shape:", df.shape)
print("Missing values:\n", df.isnull().sum())

# Step 2: Separate Features and Target
X = df.drop('Loan_Default', axis=1)
y = df['Loan_Default']

# Step 3: Identify Variable Types
numerical_cols = ['Age', 'Income', 'Credit_Score', 'Loan_Amount', '
    Debt_to_Income_Ratio']
categorical_cols = ['Employment_Type', 'Credit_Rating']
binary_cols = ['Previous_Default']

# Step 4: Train-Validation-Test Split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.5,
    random_state=42,
    stratify=y_temp
)

# Step 5: Handle Missing Values - Numerical
num_imputer = SimpleImputer(strategy='median')
X_train_num = num_imputer.fit_transform(X_train[numerical_cols])
X_val_num = num_imputer.transform(X_val[numerical_cols])
X_test_num = num_imputer.transform(X_test[numerical_cols])

# Step 6: Handle Missing Values - Categorical
cat_imputer = SimpleImputer(strategy='most_frequent')
X_train_cat = cat_imputer.fit_transform(X_train[categorical_cols])
```

```
X_val_cat = cat_imputer.transform(X_val[categorical_cols])
X_test_cat = cat_imputer.transform(X_test[categorical_cols])

# Step 7: Scale Numerical Variables
scaler = StandardScaler()
X_train_num_scaled = scaler.fit_transform(X_train_num)
X_val_num_scaled = scaler.transform(X_val_num)
X_test_num_scaled = scaler.transform(X_test_num)

# Step 8: Encode Categorical Variables
# Ordinal encoding for Credit Rating
rating_order = ['BBB', 'BBB+', 'A-', 'A', 'A+', 'AA-', 'AA', 'AA+', 'AAA']
ord_encoder = OrdinalEncoder(categories=[rating_order])
X_train_rating = ord_encoder.fit_transform(X_train_cat[:, 1].reshape(-1, 1))
X_val_rating = ord_encoder.transform(X_val_cat[:, 1].reshape(-1, 1))
X_test_rating = ord_encoder.transform(X_test_cat[:, 1].reshape(-1, 1))

# One-hot encoding for Employment Type
one_hot_encoder = OneHotEncoder(sparse_output=False)
X_train_employment = one_hot_encoder.fit_transform(X_train_cat[:, 0].reshape(-1, 1))
X_val_employment = one_hot_encoder.transform(X_val_cat[:, 0].reshape(-1, 1))
X_test_employment = one_hot_encoder.transform(X_test_cat[:, 0].reshape(-1, 1))

# Step 9: Combine Features
X_train_processed = np.hstack([
    X_train_num_scaled,
    X_train_employment,
    X_train_rating
])
X_val_processed = np.hstack([
    X_val_num_scaled,
    X_val_employment,
    X_val_rating
])
X_test_processed = np.hstack([
    X_test_num_scaled,
    X_test_employment,
    X_test_rating
])

# Step 10: Handle Class Imbalance (only on training data)
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train_processed, y_train
)

# Step 11: Save Preprocessed Data
np.save('X_train_balanced.npy', X_train_balanced)
np.save('y_train_balanced.npy', y_train_balanced)
np.save('X_val_processed.npy', X_val_processed)
np.save('y_val.npy', y_val)
np.save('X_test_processed.npy', X_test_processed)
np.save('y_test.npy', y_test)

# Step 12: Verify and Document
print("Preprocessing Complete!")
print(f"Training set shape: {X_train_balanced.shape}")
print(f"Validation set shape: {X_val_processed.shape}")
print(f"Test set shape: {X_test_processed.shape}")
print(f"Class distribution after SMOTE: {np.unique(y_train_balanced,
    return_counts=True)}")
```

Key Considerations in This Workflow:

1. **Preprocessing is applied to training, validation, and test sets appropriately**
2. **No data leakage:** Parameters learned from training set only
3. **Missing values handled appropriately for variable types**
4. **Scaling applied after train-test split**
5. **Class imbalance handled only on training data**
6. **All steps are documented for reproducibility**

This workflow ensures the model is properly trained and evaluated, avoiding common pitfalls like data leakage and improper handling of imbalanced data.